BUILD SPECIFICATION · FOR GROK BUILD

PolicyLens

A coverage-gap & risk-analysis demo inspired by Coverwatch's AI-native insurance platform.

What this is: A complete, self-contained build brief for an AI app builder to produce a small, deployed, interactive web app. It is a vertical slice of Coverwatch's product: upload a commercial insurance document → an LLM extracts structured policy data → a transparent scoring engine flags coverage gaps and risk exposure → a clean dashboard renders the result alongside a ranked carrier-match table.

Target build time: 3–4 hours end-to-end including deployment.
Deliverable: A public URL (Vercel) that an interviewer can click and interact with live.
Stack: Next.js (App Router) + TypeScript + Tailwind CSS, deployed on Vercel.

Contents

1. How to use this document

Build the app exactly to this spec. Where a number, formula, or string is given, use it verbatim — the values are tuned so the sample document produces a clean, believable result. Do not invent additional features before the acceptance checks in Section 13 all pass; those checks define "done."

Priorities, in order:

1. **It deploys and never breaks in a live demo.** The "Load sample policy" path must work instantly with zero network calls.
2. **The AI extraction works on a real uploaded PDF.** This is the headline capability.
3. **The scoring is transparent and deterministic.** Same input always yields the same output.
4. **It looks like a real product.** Clean, modern, no placeholder text.

2. Context — what Coverwatch does & why this demo lands

Coverwatch is an AI-native commercial insurance brokerage (San Francisco, 2025). Their core thesis: commercial insurance still runs on email, PDFs, and manual data entry, and they have built software that does the manual work of a brokerage. A broker normally gathers information from documents and conversations, translates it into carrier-specific applications, submits them, and chases underwriters until a policy binds.

Their product surfaces three pillars, all visible on their site:

- **Risk Assessment** — an overall score, a coverage-gap count, and risk exposure by category (property, liability, cyber, business interruption).
- **Market Comparison** — 40+ carriers ranked by a "match %" against premium, with average savings vs. the current policy.
- **Claims Tracker** — claim timeline, assigned team, documents.

> **Why PolicyLens is the right slice to build:** It hits Coverwatch's exact pain — turning an insurance PDF into structured data and an actionable risk view — and it demonstrates both halves of the candidate's target role: the **ML/AI** half (LLM structured extraction) and the **data/modeling** half (a transparent risk-scoring engine you can explain line-by-line). It is one coherent feature, not a toy, and it is small enough to deploy in an afternoon.

3. What you are building (product spec)

A single-page web app titled **PolicyLens**, sub-labelled "*Coverage intelligence for commercial insurance.*" The user journey:

1. Land on a dashboard shell with an empty state and two actions: **Upload policy (PDF)** and **Load sample policy**.
2. A small **Business profile** control (industry dropdown + annual-revenue dropdown) sits at the top; it defaults to `E-commerce / CPG` and `$1M–$5M`.
3. **Load sample policy** → instantly populates everything from a bundled, pre-parsed payload (no network call).
4. **Upload policy** → the PDF text is extracted, sent to an LLM, and returned as structured JSON, then scored identically to the sample.
5. The dashboard renders five regions: (a) stat cards, (b) risk exposure bars, (c) extracted-policy table, (d) coverage-gaps list, (e) carrier-match table.
6. Changing the business profile re-runs scoring and carrier match live, without re-uploading.

Feature list (build all of these)

#	Feature	Notes
F1	PDF upload + text extraction	Server-side; accept a single .pdf, max 10 MB.
F2	LLM structured extraction	Returns the policy JSON in Section 5's schema.
F3	Demo-safe sample loader	Pre-parsed JSON in Section 10; zero network.
F4	Risk-scoring engine	Coverage score + exposure-by-category (Section 7).
F5	Coverage-gap detection	Missing recommended coverages, with severity.
F6	Carrier-match table	12 carriers ranked by match %, with premiums (Section 8).
F7	Editable business profile	Industry + revenue; re-runs F4–F6 reactively.
F8	Dashboard UI	Section 9 layout & styling.
F9	Graceful errors	Non-insurance / unreadable PDF shows a friendly message, never a crash.

4. Architecture & tech stack

Layer	Choice	Why
Framework	Next.js (App Router) + TypeScript	One repo, server routes + UI, deploys to Vercel in one click.
Styling	Tailwind CSS	Fast, consistent, modern look.
PDF text	`unpdf` (serverless-friendly)	Reliable text extraction inside a Vercel function. Fallback: `pdfjs-dist`.
LLM	OpenAI-compatible chat API via server route	xAI Grok base URL `https://api.x.ai/v1` is OpenAI-compatible. If Grok Build injects its own model access, use that. Key stays server-side.
Scoring	Pure TypeScript module	Deterministic, testable, explainable in interview.
Hosting	Vercel	Public URL, env-var secrets, instant deploys.

REPOSITORY STRUCTURE

```
app/
  page.tsx                 // dashboard (client component)
  api/extract/route.ts     // POST: pdf bytes -> {policy JSON}
components/
  StatCards.tsx
  ExposureBars.tsx
  PolicyTable.tsx
  GapList.tsx
  CarrierTable.tsx
  ProfileControls.tsx
  UploadDropzone.tsx
lib/
  taxonomy.ts              // coverage enum + industry matrices
  scoring.ts               // coverage score + exposure (Section 7)
  carriers.ts              // carrier list + match engine (Section 8)
  sample.ts                // demo-safe sample payload (Section 10)
  llm.ts                   // LLM client + extraction prompt (Section 5)
  types.ts                 // shared TS types (Section 4 schema)
```

```
.env.local                // LLM_API_KEY, LLM_BASE_URL, LLM_MODEL
```

Security rule: The LLM API key is read only inside `api/extract/route.ts` (server). It must never appear in any client component, in the browser bundle, or committed to the repo. Verify with check S-2 in Section 13.

5. Data models & schemas

Coverage taxonomy (normalized enum)

Every extracted coverage must be mapped to exactly one of these codes. Free-text names from the document map to the closest code; anything unrecognized becomes `OTHER` with the original label preserved in `rawLabel`.

```
type CoverageCode =
  | "GENERAL_LIABILITY"
  | "COMMERCIAL_PROPERTY"
  | "BUSINESS_OWNERS_POLICY"   // BOP: satisfies GL + Property
  | "PRODUCT_LIABILITY"
  | "PROFESSIONAL_LIABILITY"   // E&O
  | "COMMERCIAL_AUTO"
  | "UMBRELLA"
  | "WORKERS_COMP"
  | "CYBER_LIABILITY"
  | "BUSINESS_INTERRUPTION"
  | "OTHER";
```

Extracted-policy schema (the LLM must return exactly this)

```
interface ExtractedCoverage {
  code: CoverageCode;
  rawLabel: string;        // original text from the document
  limit: number | null;    // per-occurrence or aggregate, USD
  deductible: number | null;
  premium: number | null;  // annual, USD, if present
}

interface PolicyData {
  namedInsured: string | null;
  carrier: string | null;
  policyNumber: string | null;
  effectiveDate: string | null;    // ISO yyyy-mm-dd
  expirationDate: string | null;   // ISO yyyy-mm-dd
  annualPremiumTotal: number | null;
  coverages: ExtractedCoverage[];
  isInsuranceDocument: boolean;    // false if the PDF is not a policy/COI
}
```

Business profile

```
interface BusinessProfile {
  industry:
    | "ECOMMERCE_CPG" | "RESTAURANTS" | "CONTRACTORS"
    | "TRUCKING" | "PROPERTY_MANAGEMENT" | "RETAIL" | "OTHER";
  annualRevenueBand: "LT_250K" | "B250K_1M" | "B1M_5M" | "B5M_10M" | "GT_10M";
  employeeCount: number;   // default 25 for the sample
}
```

Scoring result schema

```
interface Exposure { category: string; value: number; band: "Low"|"Medium"|"High"; }
interface Gap { code: CoverageCode; label: string; severity: "High"|"Medium"; why: string; }
interface ScoreResult {
  coverageScore: number;          // 0-100, higher = better posture
  gaps: Gap[];
  exposures: Exposure[];          // 4 categories
  coveragesPresentCount: number;
}
```

6. LLM extraction (the AI/ML core)

Flow inside `api/extract/route.ts`:

1. Receive the uploaded PDF as bytes. Extract raw text with `unpdf`.
2. If extracted text is < 40 characters, return `{ isInsuranceDocument:false }` (likely a scan/empty) and let the UI show the friendly message.
3. Call the LLM with the system + user prompt below, **temperature 0**, and JSON response mode.
4. Parse, validate against `PolicyData`, coerce missing fields to `null` / `[]`, and return.
5. If the LLM call fails or the key is absent, return HTTP 200 with `{ isInsuranceDocument:false, _fallback:true }` so the UI degrades gracefully and the demo still runs on the sample.

SYSTEM PROMPT (USE VERBATIM)

```
You are an insurance document parser. You convert the raw text of a
commercial insurance Certificate of Insurance (COI) or policy
declarations page into strict JSON. Map every coverage you find to one
of these codes: GENERAL_LIABILITY, COMMERCIAL_PROPERTY,
BUSINESS_OWNERS_POLICY, PRODUCT_LIABILITY, PROFESSIONAL_LIABILITY,
COMMERCIAL_AUTO, UMBRELLA, WORKERS_COMP, CYBER_LIABILITY,
BUSINESS_INTERRUPTION, OTHER. Parse dollar limits and premiums as plain
integers (no symbols or commas). Use null for anything not present. If
the text is not an insurance document, set isInsuranceDocument to false
and return empty coverages. Respond with ONLY the JSON object, no prose,
no markdown fences.
```

USER PROMPT

```
Extract this document into the schema:
{ namedInsured, carrier, policyNumber, effectiveDate (yyyy-mm-dd),
  expirationDate (yyyy-mm-dd), annualPremiumTotal,
  coverages: [{ code, rawLabel, limit, deductible, premium }],
  isInsuranceDocument }

DOCUMENT TEXT:
"""
{{rawText}}
"""
```

> **Why temperature 0 + JSON mode:** extraction must be repeatable. The interviewer may upload their own document; a deterministic, well-validated parser that handles messy real-world text is exactly the engineering signal the role wants.

7. Risk-scoring engine (exact formulas)

Implement in `lib/scoring.ts` as pure functions. All values below are authoritative.

7.1 Industry recommended-coverage matrix

E = essential, **R** = recommended. A `BUSINESS_OWNERS_POLICY` satisfies both `GENERAL_LIABILITY` and `COMMERCIAL_PROPERTY` requirements.

Industry	Essential (E)	Recommended (R)
ECOMMERCE_CPG	GENERAL_LIABILITY, PRODUCT_LIABILITY, CYBER_LIABILITY, WORKERS_COMP*	COMMERCIAL_PROPERTY, BUSINESS_INTERRUPTION, PROFESSIONAL_LIABILITY
RESTAURANTS	GENERAL_LIABILITY, COMMERCIAL_PROPERTY, WORKERS_COMP*	BUSINESS_INTERRUPTION, COMMERCIAL_AUTO, CYBER_LIABILITY
CONTRACTORS	GENERAL_LIABILITY, WORKERS_COMP*, COMMERCIAL_AUTO	COMMERCIAL_PROPERTY, UMBRELLA, PROFESSIONAL_LIABILITY
TRUCKING	COMMERCIAL_AUTO, GENERAL_LIABILITY, WORKERS_COMP*	UMBRELLA, COMMERCIAL_PROPERTY
PROPERTY_MANAGEMENT	GENERAL_LIABILITY, COMMERCIAL_PROPERTY, PROFESSIONAL_LIABILITY, WORKERS_COMP*	CYBER_LIABILITY, UMBRELLA
RETAIL / OTHER	GENERAL_LIABILITY, COMMERCIAL_PROPERTY, WORKERS_COMP*	BUSINESS_INTERRUPTION, CYBER_LIABILITY

* WORKERS_COMP is essential only when `employeeCount >= 1`.

7.2 Coverage score (0–100, higher = better)

```
score = 100
for each REQUIRED coverage for the industry that is MISSING:
    if essential:    score -= 18
    if recommended:  score -= 8
for each PRESENT coverage whose limit is non-null and below the
    industry minimum (see 7.3):           score -= 5
clamp to [0, 100]; round to integer.
```

7.3 Minimum-limit thresholds (for "underinsured" penalty)

Coverage	Min limit (USD)
GENERAL_LIABILITY	1,000,000
PRODUCT_LIABILITY	1,000,000
COMMERCIAL_PROPERTY	100,000
CYBER_LIABILITY	500,000
COMMERCIAL_AUTO	1,000,000
UMBRELLA	1,000,000

7.4 Risk exposure by category (0–100, higher = more exposure)

Compute four categories. `exposure = clamp(base − mitigation, 5, 100)`. Base values shown are for `ECOMMERCE_CPG`; for other industries use the same mitigation rules with the base table in 7.6.

Category	Base (ECOM)	Mitigation (subtract if coverage present)
Property Damage	60	COMMERCIAL_PROPERTY or BOP → −45
Liability	85	GENERAL_LIABILITY −25; PRODUCT_LIABILITY −20; UMBRELLA −15 (sum)
Cyber	80	CYBER_LIABILITY → −55
Business Interruption	70	BUSINESS_INTERRUPTION −50; else BOP present −15

7.5 Exposure band & color

Value	Band	Color
≥ 70	**High**	#dc2626 (red)

40–69	**Medium**	#d97706 (amber)
< 40	**Low**	#16a34a (green)

7.6 Base exposure table for other industries

Industry	Property	Liability	Cyber	Bus. Interruption
ECOMMERCE_CPG	60	85	80	70
RESTAURANTS	80	80	45	75
CONTRACTORS	65	88	40	55
TRUCKING	70	85	35	60
PROPERTY_MANAGEMENT	75	78	55	60
RETAIL / OTHER	70	75	55	65

7.7 Gap object generation

For each MISSING required coverage, emit a `Gap`: severity **High** if essential, **Medium** if recommended. The `why` string is a one-line plain-English reason. Examples:

```
CYBER_LIABILITY:        "Handles customer data and online payments;
                         a breach is uninsured without this."
BUSINESS_INTERRUPTION:  "No income protection if operations stop after
                         a covered loss."
GENERAL_LIABILITY:      "Third-party injury and property-damage claims
                         would be paid out of pocket."
```

Worked example (the sample document): Northwind Goods Co., E-commerce/CPG, carries GL + Product Liability + Commercial Property + Workers Comp, and is **missing Cyber Liability and Business Interruption**. Result: coverage score = 100 − 18 (Cyber, essential) − 8 (BI, recommended) = **74**. Exposures: Property 15 (Low), Liability 40 (Medium), Cyber 80 (High), Business Interruption 70 (High). Two gaps. These exact numbers should appear on screen for the sample — they are an acceptance check.

8. Carrier-match engine

Implement in `lib/carriers.ts`. Deterministic given the business profile.

8.1 Carrier list (use these 12)

```
const CARRIERS = [
  { name: "The Hartford",        rate: 1.00, appetite: ["ECOMMERCE_CPG","RETAIL","CONTRACTORS"] },
  { name: "Travelers",           rate: 1.05, appetite: ["ECOMMERCE_CPG","RESTAURANTS","PROPERTY_MANAGEMENT"] },
  { name: "Chubb",               rate: 1.18, appetite: ["ECOMMERCE_CPG","PROPERTY_MANAGEMENT"] },
  { name: "Liberty Mutual",      rate: 1.10, appetite: ["CONTRACTORS","TRUCKING","RETAIL"] },
  { name: "Nationwide",          rate: 1.08, appetite: ["RESTAURANTS","RETAIL","ECOMMERCE_CPG"] },
  { name: "CNA",                 rate: 1.12, appetite: ["PROPERTY_MANAGEMENT","CONTRACTORS"] },
  { name: "The Hanover",         rate: 1.06, appetite: ["ECOMMERCE_CPG","RETAIL"] },
  { name: "Hiscox",              rate: 1.20, appetite: ["ECOMMERCE_CPG","PROFESSIONAL"] },
  { name: "Markel",              rate: 1.22, appetite: ["RESTAURANTS","CONTRACTORS"] },
  { name: "Coalition",           rate: 1.15, appetite: ["ECOMMERCE_CPG","RETAIL"] },   // cyber-strong
  { name: "Berkshire GUARD",     rate: 1.03, appetite: ["CONTRACTORS","RETAIL","TRUCKING"] },
  { name: "Pie Insurance",       rate: 0.98, appetite: ["RESTAURANTS","RETAIL","CONTRACTORS"] },
];
```

8.2 Match % (clamp 42–96)

```
match = 55
     + (profile.industry in carrier.appetite ? 22 : 0)
     + coverageFitBonus            // 0..18, scaled by how many of the
                                   // industry's REQUIRED coverages the
                                   // current policy already satisfies
     + jitter                      // deterministic: (hashString(carrier.name
                                   //   + profile.industry) % 7) - 3
clamp(match, 42, 96)
```

8.3 Premium (USD/year)

```
revenueBase = { LT_250K:2500, B250K_1M:4000, B1M_5M:5200,
                B5M_10M:7800, GT_10M:11000 }[profile.annualRevenueBand]
premium = round( revenueBase * carrier.rate
                 * (1 + (96 - match) / 100 * 1.1) / 100 ) * 100   // round to $100
```

Sort carriers by **match descending**, tie-break by premium ascending. The #1 row gets a **BEST** badge. Show the first 12 rows.

8.4 Savings headline

```
currentPremium = policy.annualPremiumTotal ?? 6400   // sample fallback
bestPremium    = min(premium across carriers)
avgSavingsPct  = round((currentPremium - bestPremium) / currentPremium * 100)
```

Display `avgSavingsPct` as "Est. savings vs. current: NN%". If it is ≤ 0, show "Competitively priced" instead.

9. UI / UX specification

9.1 Visual language

- **Aesthetic:** clean, calm, fintech/SaaS. White background, soft shadows, rounded cards (radius ~12px), generous whitespace.
- **Palette:** ink `#16223b`, primary `#2563eb`, surface `#f7f9fc`, border `#e2e8f0`; status red/amber/green as in 7.5.
- **Type:** Inter or system sans. Big bold numbers on stat cards. Labels in uppercase 11px slate.
- **Header:** "PolicyLens" wordmark left; "Coverage intelligence for commercial insurance" subtitle; profile controls + upload on the right.

9.2 Layout (top to bottom)

1. **Toolbar:** Industry dropdown, Revenue dropdown, [Upload policy] button, [Load sample policy] button.
2. **Stat cards row (4):** Coverage Score (big number + small circular gauge, colored by band: ≥75 green, 50–74 amber, <50 red), Coverage Gaps (count), Coverages on Policy (count), Est. Savings (%).
3. **Two-column band:** left = **Risk Exposure by Category** (4 horizontal bars, each labelled with category, band word, and % filled, colored per 7.5); right = **Coverage Gaps** (list of cards: coverage name, severity chip, the `why` line).
4. **Extracted Policy panel:** header shows named insured, carrier, policy #, effective–expiration dates; then a table of coverages (Coverage, Limit, Deductible, Premium). Format money as `$1,000,000`.
5. **Carrier Match panel:** table with columns #, Carrier, Premium, Match %. Row 1 has the BEST badge. Match % rendered as a small bar or bold number. Above the table: "Est. savings vs. current: NN%".

9.3 States

- **Empty:** friendly hero prompting upload or sample.
- **Loading:** skeletton/spinner on the extracted-policy panel while the API runs (only on real upload).
- **Not an insurance doc:** inline notice "We couldn't find policy data in that PDF. Try a Certificate of Insurance or a declarations page, or load the sample." No crash, no stack trace.
- **Reactive:** changing industry/revenue recomputes scores, exposures, gaps, and carrier table immediately.

9.4 Copy rules

- No Lorem Ipsum anywhere.
- All currency uses thousands separators and a leading `$`.
- All percentages show the `%` sign.
- Dates show as `Mar 1, 2026` style.

10. Sample data (demo-safe payload)

Put both items in `lib/sample.ts`. "Load sample policy" sets state directly from `SAMPLE_POLICY` with **no fetch**. Also generate a matching `sample-coi.pdf` in `/public` (a simple text-based PDF of the COI below) so the interviewer can also test the real upload path on a known-good file.

SAMPLE COI SOURCE TEXT (ALSO RENDER THIS TO /PUBLIC/SAMPLE-COI.PDF)

```
CERTIFICATE OF LIABILITY INSURANCE          Date: 03/01/2026

NAMED INSURED: Northwind Goods Co.
1442 Harbor Way, Oakland, CA 94607

INSURER: The Hartford Financial Services Group
POLICY NUMBER: HFD-CGL-2026-558031
POLICY PERIOD: 03/01/2026 to 03/01/2027

COVERAGES
- Commercial General Liability
  Each Occurrence Limit ..... $1,000,000
  General Aggregate ..... $2,000,000
  Deductible ..... $2,500
  Annual Premium ..... $3,150
- Products / Completed Operations (Product Liability)
  Aggregate Limit ..... $2,000,000
  Annual Premium ..... $1,900
- Commercial Property
  Building & Contents ..... $350,000
  Deductible ..... $5,000
  Annual Premium ..... $1,350
- Workers' Compensation
  Statutory Limits
  Annual Premium ..... $0 (separate policy)

TOTAL ANNUAL PREMIUM ..... $6,400
```

SAMPLE_POLICY (PRE-PARSED JSON — SHIP VERBATIM)

```
const SAMPLE_POLICY = {
  namedInsured: "Northwind Goods Co.",
  carrier: "The Hartford",
  policyNumber: "HFD-CGL-2026-558031",
  effectiveDate: "2026-03-01",
  expirationDate: "2027-03-01",
  annualPremiumTotal: 6400,
  isInsuranceDocument: true,
  coverages: [
    { code:"GENERAL_LIABILITY",  rawLabel:"Commercial General Liability", limit:1000000, deductible:2500, premium:3150
},
    { code:"PRODUCT_LIABILITY",  rawLabel:"Products / Completed Operations", limit:2000000, deductible:null,
premium:1900 },
    { code:"COMMERCIAL_PROPERTY",rawLabel:"Commercial Property", limit:350000, deductible:5000, premium:1350 },
    { code:"WORKERS_COMP",       rawLabel:"Workers' Compensation", limit:null, deductible:null, premium:0 }
  ]
};
```

> **Sample must produce:** coverage score **74**; gaps = Cyber Liability (High), Business Interruption (Medium); exposures Property 15 / Liability 40 / Cyber 80 / Business Interruption 70; carrier table led by The Hartford or Pie with a BEST badge; a positive savings number. If any of these differ, the scoring/matching constants were not implemented as specified.

11. Step-by-step build plan

Phase	Tasks	~Time
0. Scaffold	Create Next.js + TS + Tailwind app. Add `lib/types.ts`, `lib/taxonomy.ts`. Commit, deploy a hello-world to Vercel to confirm the pipeline works *first*.	30m
1. Scoring core	Implement `lib/scoring.ts` and `lib/carriers.ts` exactly per Sections 7–8. Unit-test against the sample to hit score 74.	45m
2. Dashboard UI	Build components and `page.tsx` wired to `SAMPLE_POLICY`. Get the whole dashboard looking right with the sample before touching uploads.	60m
3. Upload + LLM	Build `api/extract/route.ts` with `unpdf` + the LLM prompt. Wire the dropzone. Handle the not-an-insurance-doc and fallback paths.	50m
4. Polish + profile	Profile controls re-run scoring; format money/dates; empty/loading/error states; generate `sample-coi.pdf`.	30m
5. Deploy + verify	Set env vars in Vercel, redeploy, run the full acceptance checklist on the live URL.	25m

> **Order matters:** deploy an empty app in Phase 0. Discovering a Vercel/runtime issue at hour 3.5 is the classic way these demos die. Get the sample-only dashboard fully working before adding the LLM — the demo is already viable at the end of Phase 2.

12. Deployment

1. Push the repo to GitHub; import it into Vercel (framework auto-detected as Next.js).
2. In Vercel → Project → Settings → Environment Variables, add: `LLM_API_KEY`, `LLM_BASE_URL` (e.g. `https://api.x.ai/v1`), `LLM_MODEL`.
3. Redeploy. Confirm the public URL loads and "Load sample policy" works even before the key is set.
4. Confirm `unpdf` runs in the Node serverless runtime (set `export const runtime = "nodejs"` in the route if needed).

13. Definitive acceptance checks must pass

The build is "done" only when every item passes on the deployed public URL.

BUILD & DEPLOY

[B-1] App builds with zero errors/warnings and is live at a public Vercel URL.

[B-2] Page loads in under ~3s; browser console shows no errors.

[B-3] Works on a normal laptop screen and degrades reasonably on mobile width.

SAMPLE PATH (DEMO-SAFE)

[P-1] "Load sample policy" populates the full dashboard **instantly with no network request** (verify in the Network tab).

[P-2] Coverage score reads **74**.

[P-3] Exactly two gaps show: Cyber Liability (High) and Business Interruption (Medium).

[P-4] Exposure bars read Property 15 (Low/green), Liability 40 (Medium/amber), Cyber 80 (High/red), Business Interruption 70 (High/red).

[P-5] Extracted-policy panel shows Northwind Goods Co., The Hartford, policy number, and the 4 coverages with correctly formatted limits.

UPLOAD + EXTRACTION

[X-1] Uploading `/public/sample-coi.pdf` returns structured JSON and renders the same dashboard as the sample.

[X-2] Coverages are mapped to the normalized enum (no raw free-text codes).

[X-3] Uploading a non-insurance PDF shows the friendly notice, not a crash or stack trace.

[X-4] With the LLM key absent or the call failing, the app still loads and the sample path still works (graceful fallback).

[X-5] Extraction runs at temperature 0 and returns valid JSON (no markdown fences leaking into the UI).

SCORING & MATCHING

[S-1] Scoring is deterministic: same inputs → identical outputs on repeat.

[S-2] Changing the industry or revenue dropdown recomputes score, exposures, gaps, and the carrier table live.

[S-3] Carrier table shows 12 rows sorted by match % descending; row 1 has the BEST badge; premiums fall in a plausible ~$3k–$10k range.

[S-4] "Est. savings vs. current" shows a sensible percentage (or "Competitively priced").

SECURITY & HYGIENE

[H-1] The LLM API key never appears in the client bundle or the repo (grep the build output).

[H-2] No Lorem Ipsum; all currency/percent/date formatting follows Section 9.4.

[H-3] Uploaded files are processed in memory and not logged or persisted.

INTERVIEW-READINESS

[I-1] The whole story — upload → extract → score → gaps → carriers — can be shown in under 90 seconds.

[I-2] You can open `lib/scoring.ts` and explain any number on screen from the code.

14. Interview talking points (bonus)

- **Frame it as their problem:** "Your thesis is that insurance runs on PDFs and manual entry — so I built the slice that turns a policy PDF into structured data and an actionable risk view."
- **ML half:** talk about deterministic structured extraction, schema validation, the not-a-document guardrail, and how you'd evaluate extraction accuracy (field-level precision/recall, a labelled set of COIs).
- **Modeling half:** the scoring is intentionally transparent rules, not a black box — easy to audit, easy for a broker to trust. You can then say how you'd graduate it to a learned model once you have claims/loss data.
- **Production instincts:** mention idempotent extraction, human-in-the-loop review (their actual workflow), and that real submissions are long-running and asynchronous — which is exactly why they use a durable workflow engine (Temporal). Showing you know *why* that matters is a strong signal.
- **Honesty:** be clear the carrier premiums are simulated for the demo; the real version would call carrier rating APIs / submission flows.
